

Memòria de la Pràctica 1

Agents intel·ligents adversarials i basats en cerques

Serafí Nebot Ginard
Jaume Galmés Ramis

21722 - Intel·ligència Artificial
Grau d'Enginyeria Informàtica
Curs 2025/2026

Índex

1	Introducció	1
2	Estructura de dades	2
2.1	Classe Pos i tractament de la percepció	2
2.2	Classe Estat	2
3	Algorismes basats en cerca	4
3.1	Algorisme Depth-First Search	4
3.2	Algorisme A*	4
4	Algorisme adversarial - Minimax	5
4.1	Plantejament	5
4.2	Poda α - β	6
4.3	Execució	7
5	Estudi dels resultats	8
5.1	Mesura de rendiment: Temps d'execució	8
5.2	Mesura de rendiment: Nombre de passes	9
6	Conclusió	11

Introducció

Aquesta memòria pretèn mostrar la feina realitzada sobre la pràctica 1, en les següents pàgines s'exposa l'abordament del problema des del disseny de l'estructura de dades, passant per el plantejament i implementació dels distints algorismes i acabant amb l'estudi dels resultats obtinguts.

Estructura de dades

Dins aquest apartat es descriuen les estructures de dades que hem definit per a solucionar el problema proposat. Creiem que han estat molt encetades perquè ens han ajudat a simplificar la lògica dels mètodes de cerca i a pensar sobre el problema de forma molt intuïtiva.

2.1 Classe Pos i tractament de la percepció

Per conveniència s'ha creat una classe **Pos** que extén les funcionalitats d'una tupla (**x**, **y**) i permet realitzar operacions aritmètiques i lògiques, element per element, de forma molt intuïtiva. Cosa que ha simplificat bastant el desenvolupament de la pràctica.

Per tant algunes de les percepcions com la dimensió o el torn es mantenen iguals, mentre que altres percepcions com la posició dels agents i el destí es converteixen en objectes **Pos**, de forma idèntica succeeix amb les parets, on aquestes formen un conjunt d'objectes **Pos**.

2.2 Classe Estat

Les classes **Estat** i **EstatAdv**, que permeten donar una representació de l'entorn en un moment concret i generar tot l'arbre de moviments possibles a partir d'un estat inicial. **Estat** representa un entorn d'un sol agent sobre el tauler, mentre que l'ús d'atributs addicionals donen a **EstatAdv** la capacitat de treballar sobre un entorn multiagent.

En concret, la classe **Estat** ve definit pels següents atributs:

- La posició de l'agent actual sobre el tauler
- La posició de destí final (objectiu)
- La posició dels obstacles (parets)
- La dimensió del tauler
- El camí com a seqüència d'accions que duen fins al node en qüestió
- La heurística de l'estat actual (distància Manhattan)
- El cost de l'estat actual (suma de costos del accions de tot el camí)

Per a la classe **EstatAdv** conté els següent atributs:

- La posició de l'agent actual i contrari sobre el tauler
- La posició de destí final (objectiu)
- La posició dels obstacles (parets)
- La dimensió del tauler
- El camí com a seqüència d'accions que duen fins al node en qüestió (inclouent el moviments tant de MAX i MIN)
- La distància Manhattan de l'agent actual i contrari
- El torn actual (si és de MAX o de MIN)

Algorismes basats en cerca

Els següents algorismes basats en cerca permeten, donat un estat inicial, obtenir com a solució un camí (seqüència d'accions).

3.1 Algorisme Depth-First Search

L'algorisme de cerca en profunditat o *Depth-First Search* (DFS) és un tipus de cerca no informada que recorre de forma iterada el camí descendent de pares a fills. Dins el nostre projecte aquest s'ha implementat emprant principalment un llistat dels nodes oberts i un conjunt dels nodes tancats, mitjançant l'extracció i posterior visita del darrer element de la llista de nodes oberts. A cada iteració es comprova si el node fill és l'estat objectiu.

3.2 Algorisme A*

Per contraposició a l'algorisme DFS, l'A* és un algorisme de cerca informada, és a dir, a cada iteració, s'avalua la bondat d'un node tenguent en compte no només la seva heurística sino també el cost acumulat fins al mateix.

Per a dur a terme la bondat d'un estat s'ha emprat com a heurística la distància Manhattan, és a dir:

Sigui $\mathbf{p}$ la posició de l'agent a l'estat $\mathbf{n}$ i $\mathbf{d}$ la posició de l'objectiu, amb coordenades:

$$\mathbf{p} = \begin{bmatrix} x_p & y_p \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} x_d & y_d \end{bmatrix}$$

Obtenim l'heurística com:

$$h(\mathbf{n}) = d(\mathbf{p}, \mathbf{d}) = \sum_{i=1}^2 |p_i - d_i| = |x_p - x_d| + |y_p - y_d|$$

Per tant, si el cost acumulat ve donat per la funció $c(\mathbf{n})$, el valor del node actual és:

$$f(\mathbf{n}) = h(\mathbf{n}) + c(\mathbf{n}) = d(\mathbf{p}, \mathbf{d}) + c(\mathbf{n})$$

En el cas de que dos estats tinguin el mateix valor, s'utilitza la heurística menor per a desempatar entre ells.

Algorisme adversarial - Minimax

A diferència dels algorismes de cerca informats/no informats, els algorismes adversarials ens permeten avaluar cada jugada on la solució no és el camí sino la acció òptima.

4.1 Plantejament

L'algorisme **minimax** genera, a partir d'un estat donat, totes les possibles jugades, avaluant la bondat de cada una i finalment retornant la que resulta més adient per a l'agent del torn actual. Recordem que ens interessa que l'agent MAX dugui a terme el moviment que li pugui donar més avantatge fent el supòsit de que MIN actuarà de la mateixa forma, d'aquesta manera asseguram que **minimax** es posi en el pitjor dels casos i no només contempli quan l'agent MIN jugui de forma subòptima.

Per a implementar aquest comportament, s'ha definit que la puntuació que avalua una jugada sigui com de molt s'ha aconseguit obstruir a l'agent contrari, és a dir:

Siguin a_{act} i a_{adv} l'agent del torn actual i el seu adversari, i $d(a)$ la distància Manhattan d'un agent a respecte al destí:

$$\text{puntuació}(a_{act}) = \begin{cases} d(a_{adv}), & \text{si } d(a_{act}) = 0, \\ -d(a_{act}), & \text{si } d(a_{adv}) = 0, \\ 0, & \text{en cas contrari.} \end{cases}$$

Consideram que aquesta puntuació és una millora a la de -1 si guanya MIN, 0 si no guanya ningú i 1 si guanya MAX perquè, per exemple, si MAX guanya és molt millor que ho faci si MIN queda enfora de l'objectiu que si queda aprop (MIN té menys possibilitats de guanyar). D'aquesta forma es passa més informació cap als nodes pare sobre quina de les accions és més favorable.

4.2 Poda α - β

La poda α - β permet reduir el nombre de nodes explorats. Dins el nostre projecte aquesta es veu reflexada com a dos paràmetres de la funció **minimax**.

La primera crida a **minimax** es produeix al mètode **actua** on aquest inicialitza $\alpha = -\infty$ i $\beta = +\infty$. Posteriorment les crides recursives al mètode **minimax** van confinant l'interval entre α - β fins a arribar, si s'esdevé, al compliment de la condició de poda ($\alpha \geq \beta$).

Amb la implementació de la poda α - β hem vist un increment considerable en el rendiment de l'algorisme de cerca. La següent taula mostra el una comparativa de diferents execucions de l'algorisme minimax amb poda i fora poda sobre el mateix tauler (llavor random fixada). La primera columna de cada cas és el temps i la segona el nombre de nodes generats.

Poda	0.27	3849	0.10	905	0.21	2248	2.99	13036	1.19	9965
No Poda	24.3	49863	0.74	3014	15.4	33178	54.6	82086	36.9	58230

Table 4.1: Taula comparativa: poda α - β

4.3 Execució

A continuació es presenta una descripció general del funcionament de l'algorisme **minimax**, especificant les passes que segueix durant l'execució.

A partir d'un estat passat per paràmetre:

Algorisme Minimax

Pas 1: Inicialitza $\alpha = -\infty$ i $\beta = +\infty$.

Pas 2: Si l'estat actual és un estat final, es retorna a ell mateix (conté la puntuació).

Pas 3: Genera tots els estats fills possibles.

Pas 4: Per a cada fill:

Pas 4:1. Calcula recursivament el valor de **minimax**.

Pas 4:2. Actualitza α o β segons el torn.

Pas 4:3. Si $\alpha \geq \beta$, aplica la poda i surt del bucle.

Pas 5: Retorna el valor màxim o mínim segons l'agent del torn.

Estudi dels resultats

A partir dels algorismes implementats, exposam dins aquest apartat els resultats obtinguts després d'haver duit a terme l'execució repetida dels 3 algorismes. L'experiment en qüestió prèn mesures durant 30 execucions i ens permet treure una conclusió aproximada del comportament general dels mateixos.

5.1 Mesura de rendiment: Temps d'execució

Després de dur a terme l'experiment podem comparar l'algorisme de cerca no informada DFS amb el de cerca informada A*, com veim es pot apreciar una clara millora en el temps de execució.

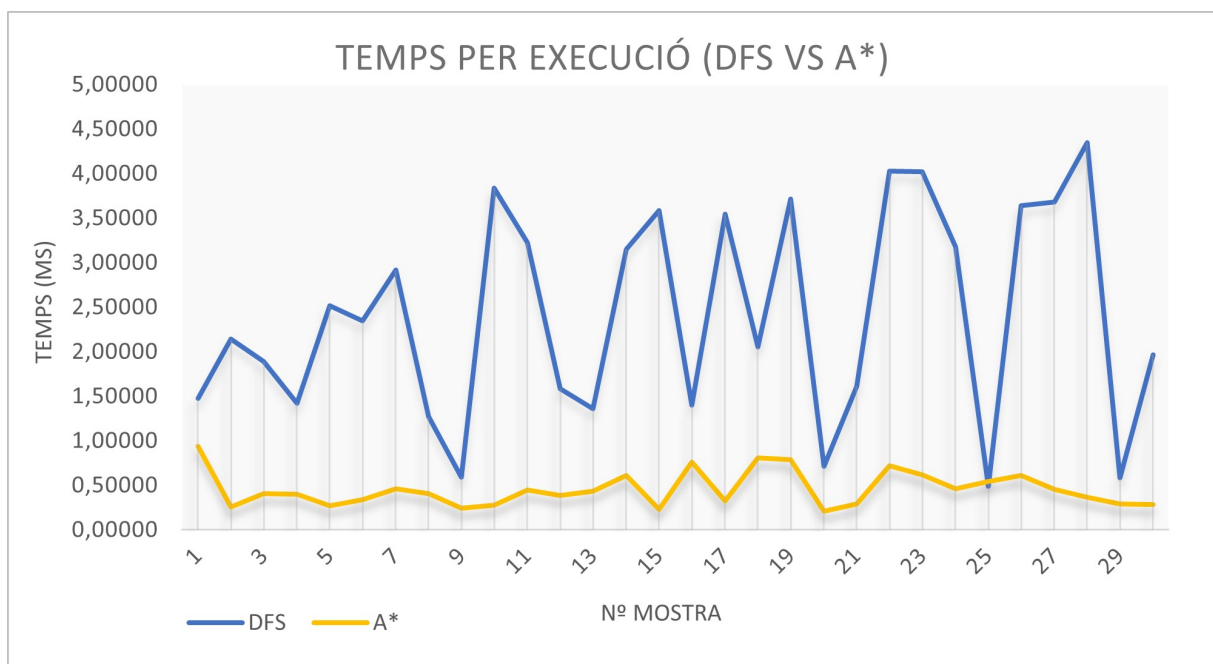


Figure 5.1: Gràfica comparativa: temps d'execució dels algorismes DFS i A*

En quant a l'algorisme minimax veim que, per qualche motiu que desconeixem, hi ha execucions que tarden un temps desorbitat, això fa que tant la mitja de l'experiment com la desviació típica es vegin alterades.

A la següent gràfica es pot observar el succès:

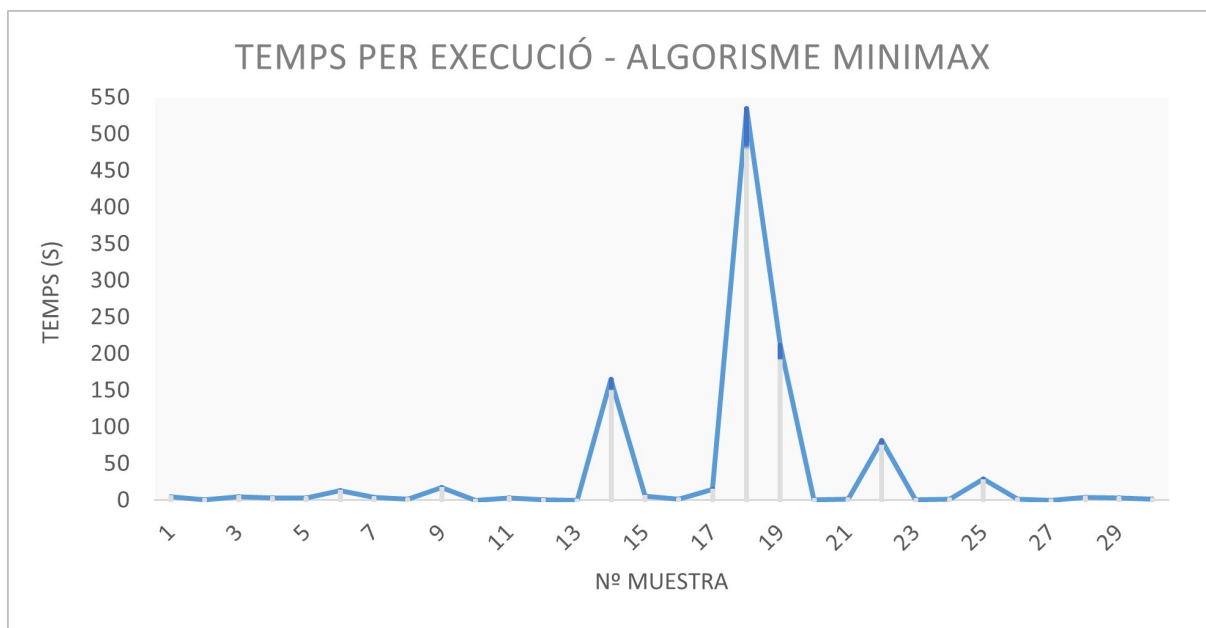


Figure 5.2: Gràfica de resultats: temps d'execució de l'algorisme minimax

A la següent taula es mostra una comparativa directa del temps mig d'execució de cada un dels 3 algorismes, així també com la desviació típica. Com podem observar les mostres anòmales del minimax afecten directament sobre els resultats globals.

Algorisme	Mitja	Desviació típica
DFS	0,002409387	0,001169715
A*	0,000454857	0,000192119
MINIMAX	37,13	104,1104874

Table 5.1: Taula comparativa: temps d'execució

5.2 Mesura de rendiment: Nombre de passes

Una altra mesura de rendiment que hem considerat que pot ser interessant estudiar és el nombre de passes que tarda un algorisme en qüestió en arribar a l'objectiu.

A la Figura 5.3 trobam la comparació directa entre els algorismes de cerca DFS i A*. Recordem que la cerca informada ens permet obtenir un camí òptim, per tant, resulta evident que el nombre de passes es veurà reduït al descartar i no seguir explorant camins que ja son massa costosos de per si.

Per altra banda a la Figura 5.4 trobam el nombre de passes de l'algorisme minimax i la comparativa numèrica dels 3 algorismes a la Taula 5.2. Com es pot observar tot i tenir variacions dràstiques de les medicions de minimax, seguim obtenint un valor bastant contingut amb una desviació no tan dolenta.

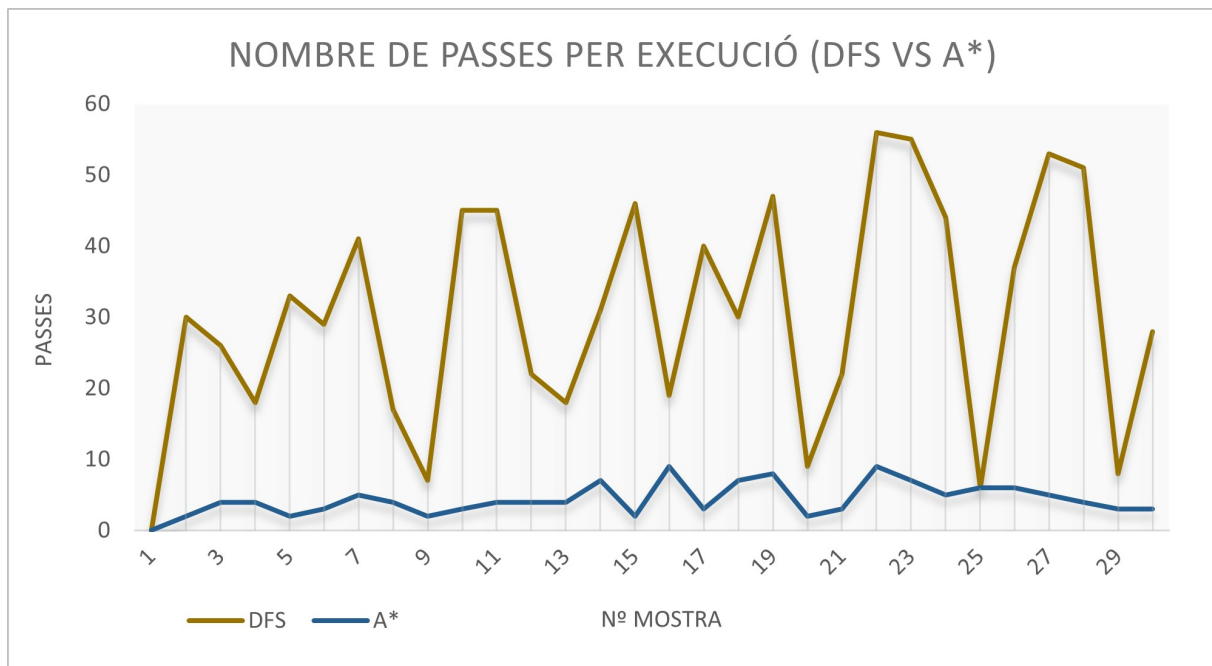


Figure 5.3: Gràfica comparativa: DFS i A*

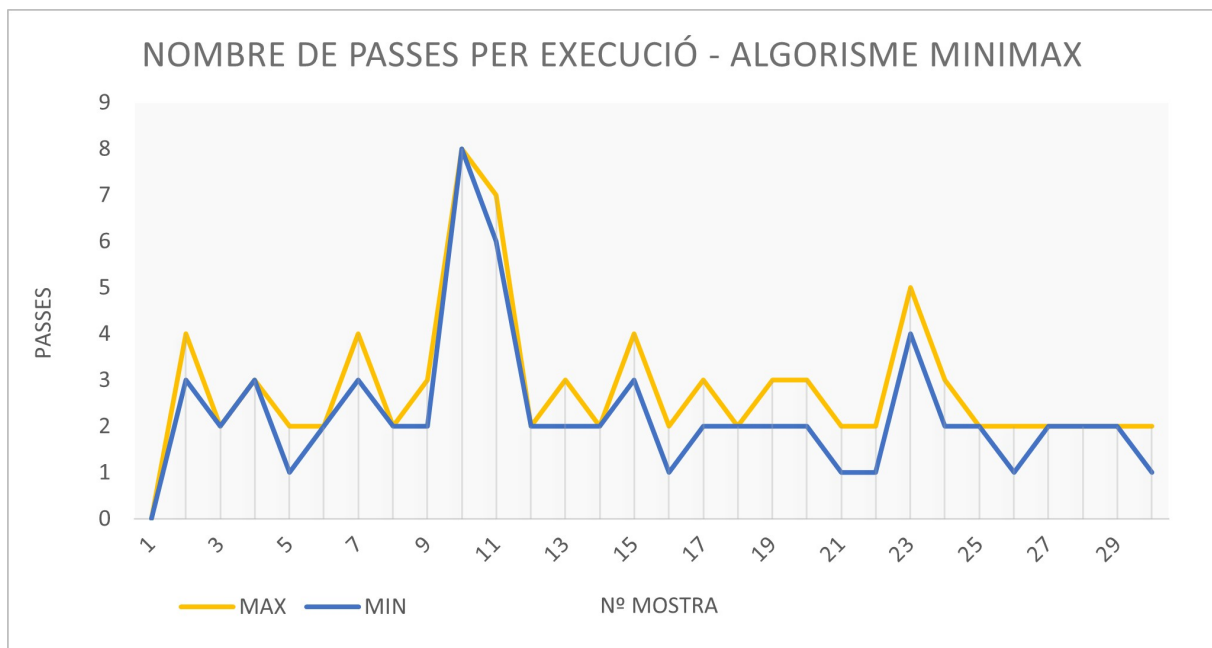


Figure 5.4: Gràfica de resultats: Algorisme Minimax

Algorisme	Mitja	Desviació típica
DFS	31,482758621	14,8964719
A*	4,482758621	2,061480714
MINIMAX (MAX)	2,93	1,48396102
MINIMAX (MIN)	2,34	1,468657892

Table 5.2: Taula comparativa: Nombre de passes per execució

Conclusió

La pràctica realitzada ha permès aplicar els conceptes i tècniques apresos al llarg del curs d'intel·ligència artificial per al desenvolupament d'agents basats en cerques i agents adversarials. Mitjançant la implementació dels algorismes de cerca no informada, A* i Minimax amb poda α - β , hem pogut observar de primera mà com cada tècnica aborda el problema de trobar un camí cap a l'objectiu i com afecta la seva eficiència i rendiment segons les restriccions del taulell i les accions disponibles.

A més, la pràctica ha posat de manifest la importància de la representació adequada dels estats, la gestió de les accions i el càlcul de puntuacions en entorns multiagent. L'anàlisi comparatiu dels algorismes implementats ha evidenciat les diferències entre cerques informades i no informades, així com la rellevància de la poda α - β per optimitzar la presa de decisions en situacions adversarials.

En conclusió, aquesta pràctica no només ha reforçat els coneixements teòrics adquirits a classe, sinó que també ha desenvolupat habilitats de programació, anàlisi i documentació de solucions en un entorn controlat, proporcionant una base sòlida per a futurs treballs en intel·ligència artificial aplicada.

Pràctica Bloc I

Intel·ligència Artificial

Agents intel·ligents adversarials
i basats en cerca

Serafí Nebot Ginard
Jaume Galmés Ramis

Grau d'Enginyeria Informàtica
Curs 2025/2026